



武汉理工大学

WUHAN UNIVERSITY OF TECHNOLOGY

课程名称	企业级应用软件设计与开发
项目名称	SuperFoodie 超级吃货智能助手
方向	Agentic AI 原生开发
学号	2025302933
姓名	杨继昆
专业	计算机技术
指导教师	戚欣
提交日期	2026 年 6 月 22 日

目录

一、选题背景与设计思想 3

 1.1 问题定义 3

 1.2 现有方案不足 3

 1.3 项目价值 4

 1.4 技术路线 4

二、Specs 规格文档 5

 2.1 Product Spec: 明确用户场景与产品目标 5

 2.2 Architecture Spec: 明确模块职责与数据流 5

 2.3 API Spec: 约束前后端通信格式 6

 2.4 测试与评估 Spec: 证明系统可运行 6

 2.5 Specs 对实现过程的影响 6

三、系统架构与设计 8

四、关键实现与代码展示 11

五、测试与评估 14

六、系统升级与扩展 19

 6.1 可扩展架构 19

 6.2 下一阶段计划 19

 6.3 AI 能力演进路径 19

七、课程总结 21

 7.1 个人收获 21

 7.2 工程思维转变 21

 7.3 对课程的建议 21

一、选题背景与设计思想

1.1 问题定义

本项目选择“智能饮食推荐”作为应用场景，是因为饮食决策具有多约束、多场景和强主观特征。用户的真实需求往往是口语化和不完整的，例如“想吃辣的”“两个人吃点清淡的”“附近找个不贵的店”“家里只有鸡蛋和青菜能做什么”。这些输入背后同时包含口味、人数、预算、位置、时间、健康限制、食材可得性和信息可信度等因素。

如果系统只返回泛泛建议，用户仍然需要继续搜索菜谱、核对食材、判断做法是否可靠、确认是否符合忌口，并寻找图片或餐厅信息，无法真正降低决策成本。SuperFoodie 试图解决的不是单纯生成菜名，而是把自然语言需求转化为可执行、可验证、可展示的推荐流程。

在家庭做饭场景中，系统需要给出候选菜、推荐理由、食材、调料、步骤和图片；在外出就餐场景中，系统需要结合位置、距离、人均、评分和用户偏好给出餐厅建议。最终结果还可以导出为 PDF 报告，方便保存、分享和二次确认。

因此，本项目强调从“聊天式回答”转向“任务式推荐”。系统不仅要理解用户说了什么，还要判断哪些信息缺失、哪些约束需要优先满足，以及最终结果能否直接用于做饭、点餐或与他人讨论。

1.2 现有方案不足

现有饮食推荐方案主要包括搜索引擎、菜谱 App、外卖/点评平台和通用聊天机器人，但它们在完整性、个性化和工程可靠性方面仍存在不足。搜索引擎信息分散，用户需要自行判断内容质量；菜谱 App 虽然提供步骤和图片，但上下文能力较弱，历史偏好难以稳定影响推荐；外卖和点评平台更适合餐厅检索，却难以覆盖家庭做饭场景，也不容易解释推荐逻辑。

通用聊天机器人可以生成较完整的文本，但如果不接入外部工具和结构化校验，容易出现步骤不可验证、图片缺失、来源不清楚、忌口处理不严格、JSON 格式不稳定等问题。从工程角度看，一次性让模型生成多个完整菜谱会增加 token 压力，也更容易出现字段缺失、内容截断和解析失败。

外部平台的不稳定也是实现中的重要风险。图片检索、地图接口和小红书内容都可能受到网络、权限、Cookie 或限流影响。如果系统缺少超时、重试和兜底策略，任何一个节点异常都可能拖慢甚至中断整条推荐链路。

因此，本项目将推荐链路拆分为“快速规划菜名”和“并发补全详情”两个阶段。第一阶段快速理解用户意图并生成结构化候选方案，第二阶段再并发补全菜谱详情、图片、外部参考和推荐理由，从而降低单次模型输出压力，提高响应速度和容错能力。

1.3 项目价值

本项目的价值主要体现在三个层面。第一是用户价值：把“吃什么”这个开放、模糊且高频的问题，转化为一组可比较、可确认、可执行的推荐结果，减少用户在多个 App 和网页之间反复切换的成本。

第二是工程价值：本项目不是简单的 Prompt Demo，而是将 LLM、向量检索、知识规则、外部 API、Agent 工作流和前端展示整合为一个可运行系统，覆盖自然语言理解、任务拆解、工具调用、并发执行、异常处理、结构化输出和报告生成等工程问题。

第三是课程价值：项目覆盖 Product Spec、Architecture Spec、API Spec、Agent 状态机、工具定义、测试评估和部署扩展，能够展示一个智能应用从需求分析到系统实现的完整过程。饮食推荐场景贴近日常生活，也便于通过实际结果进行功能验证和用户体验评估。

同时，该场景的结果比较容易被用户直观判断。推荐是否符合口味、步骤是否可执行、图片是否匹配、餐厅距离是否合理，都可以通过 Demo 和实际体验进行验证，因此适合作为课程项目展示。

1.4 技术路线

本项目采用前后端分离架构，并结合 Agentic Workflow 实现智能推荐流程。前端使用 React 负责用户输入、结果卡片、图片展示和 PDF 导出入口；后端使用 FastAPI 统一管理请求处理、会话状态、Agent 调用、外部接口适配、异常处理和报告生成。

Agent 层负责意图识别、任务规划、工具选择、外部调用和结果合并。系统首先判断用户需求属于家庭做饭、外出就餐还是混合型咨询，再选择对应工具链。数据层结合 Graph-RAG、Milvus 和本地模板，提供饮食知识、偏好记忆、安全规则和结构化输出约束。

外部工具通过 MCP 或适配层接入，包括小红书内容搜索、Tavily 图片检索和高德地图餐厅检索，用于补充真实图片、生活化内容和地理位置信息。整体技术路线强调模块化、可扩展和可降级，后续可继续扩展营养分析、购物清单、多人协同点餐和移动端适配等功能。

这种技术路线的核心是把模型能力和工程约束结合起来：模型负责理解和生成，工具负责获取真实世界信息，规则与模板负责校验边界，前端和报告模块负责把结果转化为用户可理解、可保存的形式。

二、Specs 规格文档

Specs 文档是本项目从想法走向工程实现的关键环节。对于基于大模型和多工具调用的智能应用，如果只停留在“让模型推荐食物”的描述，容易导致边界模糊、接口频繁变化和前后端联调困难。因此，本项目通过 Product Spec、Architecture Spec、API Spec 和测试与评估 Spec 提前约束产品目标、系统架构、通信格式和验证标准。

Product Spec 主要回答“系统为谁解决什么问题”，Architecture Spec 说明“系统由哪些模块组成”，API Spec 约束“前后端如何交换数据”，测试与评估 Spec 用于证明“系统是否真的可运行”。这些文档共同构成 SDD 的核心内容，使项目不只是 Prompt Demo，而是具备明确工程结构的智能应用。

在开发过程中，Specs 文档还承担了回归检查的作用。当功能调整或接口变化时，可以根据文档判断修改是否仍然符合最初的用户场景、模块职责和数据格式，避免项目在反复试错中偏离目标。

2.1 Product Spec: 明确用户场景与产品目标

Product Spec 用于定义用户画像、核心场景、交互流程和输出形态。SuperFoodie 面向日常饮食选择困难的普通用户，因此系统将场景划分为家庭做饭和外出就餐两类：前者关注菜品、食材、调料、步骤、推荐理由和图片，后者关注位置、距离、人均、评分、餐厅类型和推荐理由。

在输入设计上，系统允许用户直接使用自然语言表达需求，而不要求填写复杂表单；在输出设计上，推荐结果以卡片形式呈现，便于用户比较、保存和分享。Product Spec 还定义了 PDF 导出功能，使推荐结果能够转化为可保存、可传播的饮食决策文档。

2.2 Architecture Spec: 明确模块职责与数据流

Architecture Spec 用来约束系统结构，避免所有逻辑集中在一个接口或一个 Prompt 中。本项目整体包括前端展示层、FastAPI 服务层、Agent 编排层、数据与知识层、外部工具层和报告生成层。React 负责交互和展示，FastAPI 负责会话、异常、图片代理和报告生成，Agent 层负责意图识别、任务规划、工具选择和结果合并。

数据与知识层结合 Graph-RAG、Milvus 和本地模板，用于组织食材、口味、健康限制、偏好记忆和输出结构。小红书、Tavily、高德地图等外部工具通过 MCP 或适配层接入，弥补大模型在实时信息、真实图片和地理位置数据方面的不足。

通过 Architecture Spec，系统可以把可替换部分和稳定边界区分开。例如外部搜索工具可以更换，模型也可以切换，但前端依赖的推荐字段、会话状态和报告数据结构应保持稳定。

2.3 API Spec：约束前后端通信格式

API Spec 是前后端联调稳定性的基础。核心接口包括 POST /api/foodie/start、会话状态查询、推荐结果返回、图片代理接口和 PDF 导出接口。规格文档明确请求字段、响应字段、字段类型和异常返回格式，避免开发过程中字段随意变化。

推荐结果字段是 API Spec 的重点。菜品推荐包括 name、menu_slot、reason、avoid_reason、ingredients、seasonings、steps、image_url 等字段；餐厅推荐包括 name、address、distance、avg_price、rating、category 和 reason 等字段。固定字段后，前端卡片和 PDF 模块可以复用同一份结构化数据。

API Spec 还减少了前后端协作中的不确定性。当前端按照固定字段开发卡片和报告模板后，后端只要保持字段稳定，就可以继续优化 Agent 内部逻辑，而不会频繁影响页面展示。

2.4 测试与评估 Spec：证明系统可运行

测试与评估 Spec 用来验证系统是否真正可用，而不是只看页面能否展示。功能测试覆盖家庭做饭、外出就餐、图片展示和 PDF 导出；Agent 行为评估关注任务拆解是否合理、是否遵守安全规则、是否能在失败时兜底；Benchmark 指标则记录响应时间、JSON 解析成功率、图片补全成功率和 PDF 生成成功率等内容。

这些测试内容能够为最终报告提供证据。相比只描述系统功能，截图、日志和指标更能说明系统已经完成从用户输入到结构化推荐、外部增强和报告导出的完整闭环。

2.5 Specs 对实现过程的影响

在实现过程中，Specs 文档推动了推荐链路的调整。最初系统尝试让模型一次性生成多个完整菜谱，但实际运行中容易出现 JSON 字段缺失、内容截断和解析失败。因此，第二轮实现改为“两阶段生成”：第一阶段只返回 name、menu_slot、search_keywords、reason、avoid_reason 等紧凑字段，第二阶段再并发补全食材、步骤、图片和外部参考。

这个变化也影响了后续架构图和代码展示。Agent 不再是单一的大模型调用，而是被拆分为 Planner、Detail Enricher、Image Fetcher、Result Merger 和 Report Builder 等步骤。由此可见，Specs 文档不仅是前期设计材料，也会在实现过程中持续推动系统架构优化。

规格文档	核心内容	在项目中的作用
Product Spec	用户画像、家庭做饭/外出就餐场景、输入字段、结果卡片、PDF 导出	保证系统围绕用户任务展开，而不是只做模型问答
Architecture Spec	前端、FastAPI、Agent 编排、Graph-RAG、Milvus、MCP、外部	保证系统可维护、可替换、可扩展

	API 的职责划分	
API Spec	POST /api/foodie/start、会话状态、推荐结果字段、图片代理和 PDF 导出字段	保证前后端联调稳定，避免字段随意变化
测试与评估 Spec	功能测试、Agent 行为评估、失败降级、Demo 截图和 Benchmark 口径	保证最终报告不是只描述功能，而是能证明功能可运行

三、系统架构与设计

本章重点展示 SuperFoodie 的系统架构、Agent 交互流程和数据流设计。架构设计的目标不是简单把大模型接入前端，而是把用户输入、任务规划、外部工具、知识检索、结构化输出和 PDF 报告组织成一条稳定链路。通过分层设计，前端只负责交互体验，后端负责服务边界和状态管理，Agent 层负责决策与工具编排，数据层负责记忆、规则和安全约束。因此，系统设计时重点考虑了三个问题：第一，用户输入往往是不完整的，需要由 Agent 进行意图识别和条件补全；第二，外部工具可能出现超时、Cookie 失效或返回为空，主流程必须具备降级能力；第三，推荐结果最终要进入前端卡片和 PDF 报告，所以所有中间结果都需要尽量保持结构化。

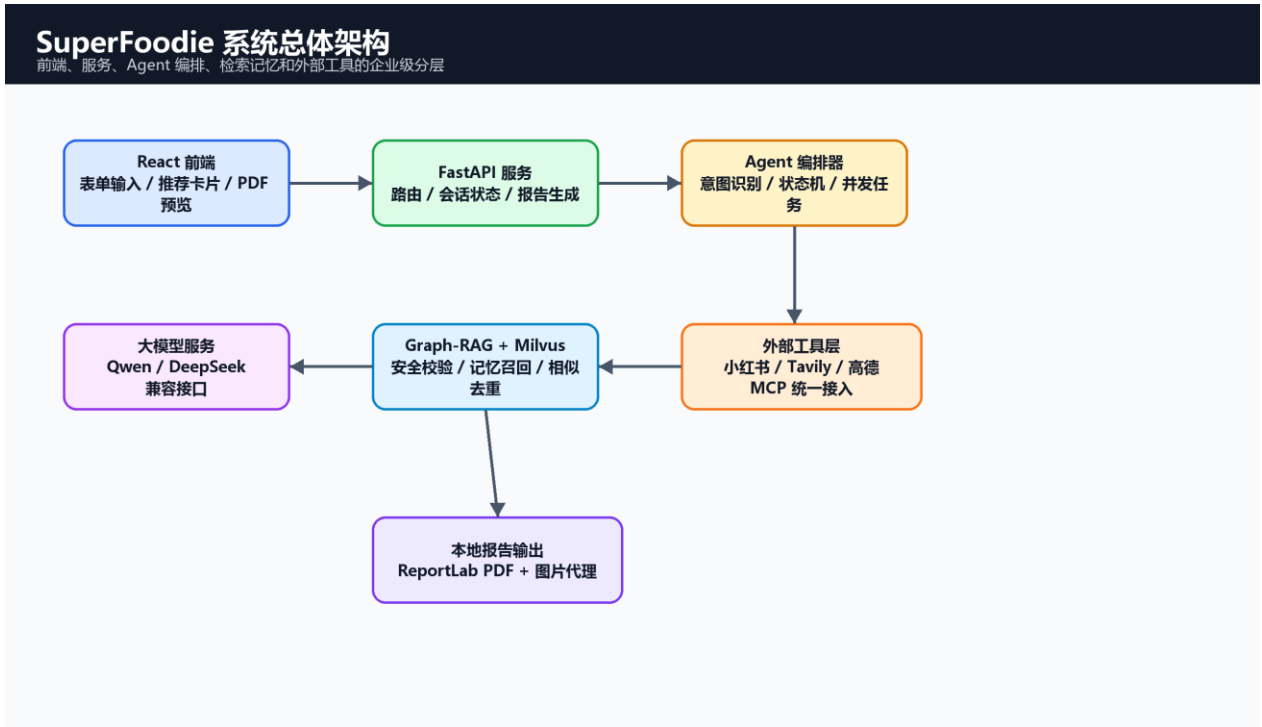


图 1 SuperFoodie 系统总体架构

总体架构采用清晰分层：React 负责体验，FastAPI 负责服务边界，Agent 编排器负责决策和任务调度，Graph-RAG 与 Milvus 负责安全和记忆，外部工具负责真实世界数据补充，ReportLab 负责最终报告输出。

这种分层方式使每个模块的职责比较明确。React 不直接访问高德、小红书或 Tavily，避免前端暴露密钥和平台细节；FastAPI 作为统一入口处理会话、异常和返回格式；Agent 编排器根据家庭做饭或外出就餐选择不同工具链；Graph-RAG 和 Milvus 则为推荐提供规则约束和历史偏好支撑。

Agent 交互流程

更新后的家庭做饭链路：小红书探测与菜名规划并行，详情阶段并发补全

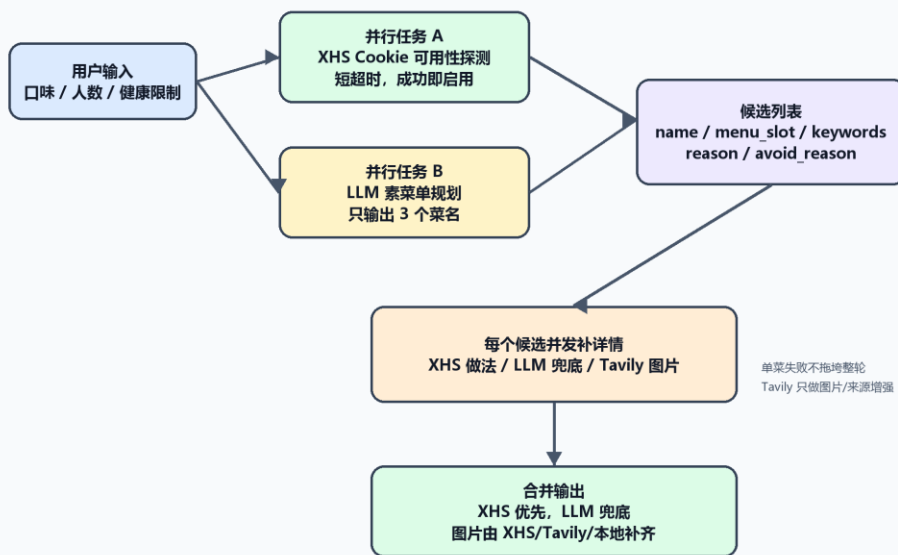


图 2 Agent 交互流程：XHS 探测与菜名规划并行

更新后的 Agent 流程把原来的一次长生成拆成两段。第一段并行启动 XHS 可用性探测和 LLM 菜名规划；第二段对三个候选菜并发补全详情。这样既减少了第一轮 JSON 截断风险，也让小红书、LLM 和 Tavily 的耗时可以重叠。

在这个流程中，Planner 只承担“确定方向”的任务，不负责生成全部菜谱细节；Detail Enricher 再根据菜名补全食材、调料和步骤；Image Fetcher 负责图片增强；Result Merger 负责字段合并和兜底。拆分之后，即使某一个外部增强步骤失败，系统仍然可以保留基础推荐结果。

数据流设计

用户输入、记忆检索、外部内容、结构化推荐与 PDF 输出的数据闭环

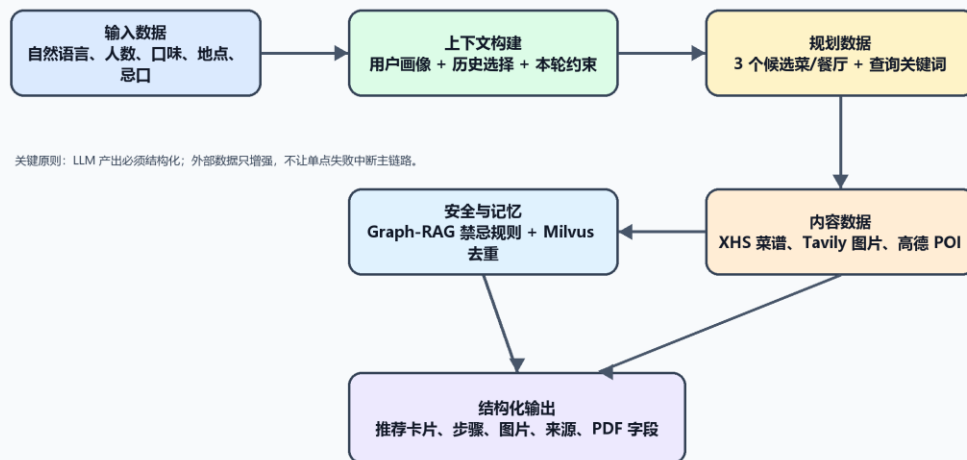


图 3 数据流设计：从用户输入到结构化推荐和 PDF

数据流的核心原则是结构化和可降级。LLM 输出必须符合 schema；小红书和 Tavily 只作为增强数据源，不允许单点失败中断主链路；Graph-RAG 与 Milvus 在推荐前后都参与安全校验与重复控制。

从数据流角度看，用户输入首先被转化为结构化任务，再进入推荐规划、工具调用和结果合并阶段。最终输出不仅服务于页面展示，也会被 PDF 模块复用。因此字段命名、图片地址、推荐理由和失败提示都需要保持一致，避免前端展示、后端日志和报告内容之间出现不一致。

四、关键实现与代码展示

关键实现围绕 Agent 核心循环、工具定义、配置文件、并发补全和 AI IDE 协作展开。代码展示采用 PlanetB 风格高亮图片，避免 Word 中直接粘贴代码造成缩进和字体混乱。本章更关注代码背后的工程思路：如何把一次自然语言请求拆成多个可执行步骤，如何让外部工具失败时不影响主流程，以及如何让模型输出稳定进入前端页面和 PDF 报告。

Agent 核心循环：规划、工具调用、合并与降级

PlanetB 风格高亮，用于 Word 代码展示

```
1  async def run_foodie_agent(request: FoodieRequest) -> FoodieResponse:
2      context = build_context(request)
3      intent = await classify_intent(context)
4      safety = await graph_rag_guard(context)
5
6      if intent == "home_cooking":
7          return await run_home_recipe_pipeline(context, safety)
8      if intent == "dining_out":
9          return await run_dining_pipeline(context, safety)
10     return ask_followup_question(context)
```

代码展示 1 Agent 核心循环

Agent 核心循环并不是让模型一次性返回最终答案，而是先构建上下文，再识别意图，随后进入不同业务管道。家庭做饭和外出就餐共享安全校验、会话状态和报告输出，但具体工具链不同。

这样设计可以降低主函数复杂度。Agent 核心循环只负责判断任务类型和调度流程，具体的菜谱生成、餐厅搜索、图片检索和报告生成由独立函数完成。后续如果增加营养分析或购物清单，也可以作为新的工具节点接入，而不需要重写整个推荐入口。

工具定义：外部平台能力统一为可超时任务

PlanetB 风格高亮，用于 Word 代码展示

```
1  @tool(timeout=3.0, retry=0)
2  async def probe_xhs_available(query: str) -> bool:
3      logger.info("xhs_probe.start query=%s", query)
4      result = await aione.xhs_search(keyword=query, limit=3)
5      return bool(result and result.items)
6
7  @tool(timeout=8.0, retry=1)
8  async def search_recipe_images_online(keyword: str) -> ImageBundle:
9      return await tavily.search_images(keyword)
```

代码展示 2 工具定义：统一超时和降级策略

工具定义层把小红书、Tavily、高德等外部能力包装成可超时、可重试、可观测的任务。这样业务代码不直接依赖某个平台的调用细节，也能在 Cookie 失效或网络超时时快速降级。

工具层还统一处理返回字段和异常格式。比如图片检索失败时返回占位结果，地图接口超时时返回明确的错误原因，小红书不可用时只影响参考内容，不影响基础菜谱生成。通过这种封装，Agent 可以专注于任务编排，而不是在业务逻辑中反复处理平台差异。

```
配置文件：模型、工具和降级策略可切换  
Planet8 风格高亮，用于 Word 代码展示  
  
1 LLM_PROVIDER=dashscope  
2 LLM_MODEL=qwen3.5-plus  
3 ENABLE_THINKING=false  
4  
5 XHS_PROBE_TIMEOUT_SECONDS=3  
6 RECIPE_DETAIL_TIMEOUT_SECONDS=12  
7 TAVILY_IMAGE_TIMEOUT_SECONDS=6  
8 FALLBACK_IMAGE_DIR=assets/recipe_placeholders
```

代码展示 3 配置文件：模型与工具策略可切换

配置文件把模型、思考模式、外部工具超时、本地兜底图片等策略集中管理。本项目在测试 DeepSeek 与 Qwen3.5-plus 时，能够通过模型名和 enable_thinking 参数切换运行方式，避免把实验参数写死在业务代码里。

配置集中化也方便进行对比实验。不同模型、不同超时时间和不同图片策略会直接影响响应速度与稳定性，如果这些参数散落在代码中，后续调试和复现实验都会比较困难。将它们放入配置文件后，系统可以更容易切换运行方案，也便于在报告中说明实验条件。

```
详情阶段：三个候选并发补全，慢任务不阻塞整轮  
Planet8 风格高亮，用于 Word 代码展示  
  
1 detail_tasks = [  
2     complete_one_recipe(candidate, xhs_available)  
3     for candidate in menu_candidates  
4 ]  
5  
6 recipes = await asyncio.gather(  
7     *detail_tasks,  
8     return_exceptions=False,  
9 )  
10 response = build_recommendation_response(recipes)
```

代码展示 4 详情阶段并发补全

详情阶段并发补全是本项目稳定性提升最明显的部分。第一阶段只生成紧凑菜名规划，第二阶段再对每个候选菜并发补全食材、步骤、图片和参考来源。这样既减少单次 LLM 输出长度，也能让图片检索和内容补全同时进行，降低用户等待时间。

AI IDE 在本项目中主要用于快速定位调用链、生成局部重构方案、解释日志和补充测试。传统调试主要看断点和堆栈，而 Agent 应用还需要看 Prompt、工具输入输出、模型返回

JSON、外部接口耗时和降级路径。开发方式因此从“只调代码”扩展为“调代码、调上下文、调工具链和调评测样例”。本次所使用的 AI IDE 为 Codex。



图 4 使用 Codex 辅助项目修改与联调

五、测试与评估

测试与评估分为功能测试、Agent 行为评估、Benchmark 观察和 Demo 截图四部分。由于系统依赖 LLM 和外部平台，测试不能只看一次成功结果，还要关注结构化输出是否稳定、工具失败后是否能降级、前端展示是否完整，以及 PDF 报告是否与页面结果一致。

功能测试主要验证家庭做饭和外出就餐两条主链路。家庭做饭场景需要检查菜名、推荐理由、食材、步骤和图片是否完整；外出就餐场景需要检查餐厅名称、距离、人均、评分和地址是否能正确展示。对于用户输入不完整的情况，系统应能够给出合理默认值，而不是直接失败。

评估维度	测试内容	结果与说明
功能测试	家庭做饭、外出就餐、PDF 生成、图片代理、推荐卡片展示	核心流程可运行，页面能展示 3 个候选和详情
Agent 行为评估	意图识别、忌口处理、候选数量、步骤结构化、失败降级	候选规划改为紧凑 schema 后，JSON 截断风险明显降低
Benchmark	比较一次性完整生成与两段式并发生成的等待结构	两段式方案更利于流式展示和慢任务隔离，后续可用日志持续量化
Demo 截图	输入页、推荐页、详情页、外出就餐页、PDF 预览	截图已插入报告，证明系统不是静态设计稿

功能测试关注系统是否能完成用户任务；Agent 行为评估关注系统是否按预期调用工具、是否遵守安全规则、是否在失败时兜底；Benchmark 关注不同链路设计对等待时间和稳定性的影响。当前版本已经记录 xhs_probe、menu_plan、recipe_detail、tavily_image 等节点日志，可以观察每个节点的耗时、成功状态和失败原因。

从测试结果看，两阶段生成比一次性生成更容易保持 JSON 稳定。即使图片检索或小红书参考失败，基础菜谱仍然可以返回；如果地图接口不可用，系统也可以保留用户输入和错误提示，避免前端长时间空白。Demo 截图部分用于展示这些功能已经连接成完整流程，包括输入、推荐卡片、详情页、地图结果和 PDF 报告。

SuperFoodie

超级吃货智能决策引擎·纯向量记忆与 Graph-RAG 图谱驱动

1. 设置吃货偏好

吃货学号/用户名

user_cs599_test

刷新

美食路径选择

自己做(家庭主厨指导)

就餐人数

1 人食(单人餐精简分量)

近期身体状况(感冒咳嗽、痛风等, Graph-RAG 依据此警示)

正常健康

今日想做菜系或现有食材(可不填, 系统会自动推荐)

辣的

启动美食决策

图 5 首页与输入表单

SuperFoodie

超级吃货智能决策引擎·纯向量记忆与 Graph-RAG 图谱驱动

1. 设置吃货偏好

吃货学号/用户名

user_cs599_test

刷新

美食路径选择

出去吃(商圈探店决策)

就餐人数

1 人食(单人餐精简分量)

近期身体状况(感冒咳嗽、痛风等, Graph-RAG 依据此警示)

正常健康

目标商圈/起点(可提前规划)

维佳 - 文慧街与文昌西路交叉口

选商圈

人均预算限制(元)

100

想吃菜系/口味

川菜

启动美食决策

图 6 用户输入示例

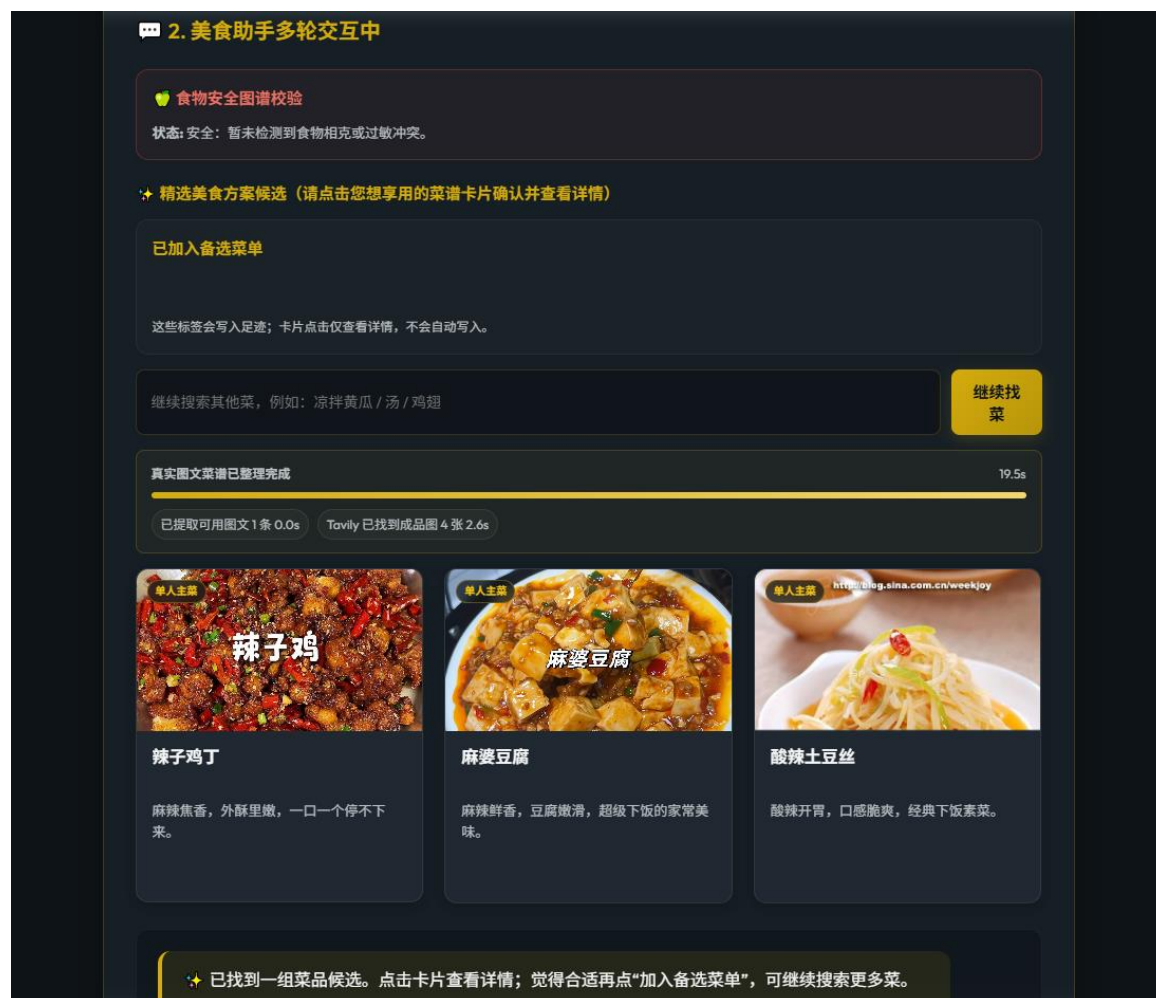


图 7 家庭做饭推荐结果



图 8 菜谱详情与图片展示

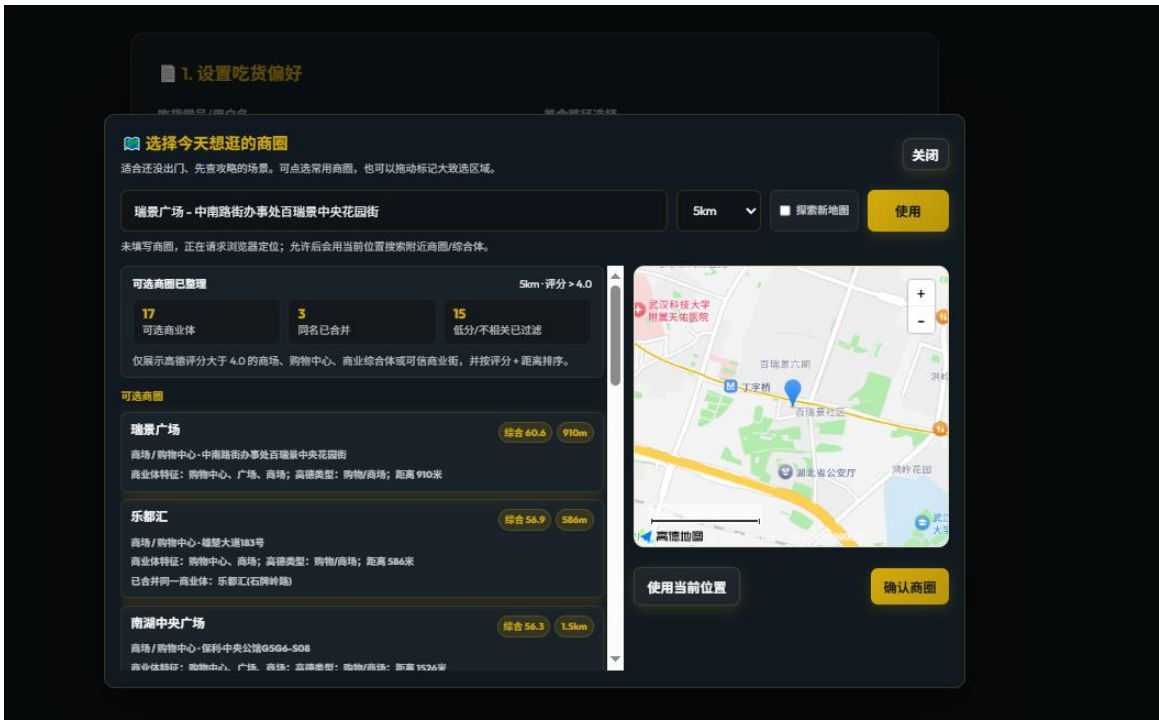


图 9 外出就餐地图推荐

菜品信息与烹饪步骤

1. 麻辣豆腐



菜品说明：麻辣鲜香，豆腐嫩滑，超级下饭的家常美味。

食材：1. 嫩豆腐 400 克
2. 猪肉末 100 克
3. 蒜苗 2 根

调料：1. 郫县豆瓣酱 1.5 汤匙
2. 花椒粉 1 茶匙
3. 生抽 1 汤匙
4. 老抽 半茶匙
5. 白糖 1 茶匙
6. 淀粉 1 汤匙
7. 食用油 适量
8. 姜蒜末 适量

做法步骤：1. 嫩豆腐切成 2 厘米见方的小块，放入加了少许盐的沸水中焯烫 1 分钟，捞出沥干备用。
2. 热锅凉油，下入肉末煸炒至变色发白，加入姜蒜末和郫县豆瓣酱炒出红油。
3. 加入适量清水煮开，调入生抽、老抽和白糖，轻轻滑入豆腐块，中小火煮 3 分钟入味。
4. 分两次淋入水淀粉勾芡，使汤汁浓稠包裹住豆腐，撒上切好的蒜苗段。
5. 出锅装盘，表面均匀撒上一层花椒粉即可享用。

饮食安全提示

安全：暂未检测到食物相克或过敏冲突。

图 10 PDF 报告预览

六、系统升级与扩展

6.1 可扩展架构

从当前项目状态看，系统已经具备继续扩展的基础：前端、FastAPI、Agent 编排、外部工具和报告输出之间边界相对清晰；小红书、Tavily、高德都可以作为工具插件替换；Graph-RAG 和 Milvus 也可以独立演进。后续如果增加新的平台，例如大众点评、盒马菜谱、营养数据库或学校食堂数据，不需要重写主流程，只需要补充工具适配器和字段归一化逻辑。扩展时需要重点保持接口稳定。新增工具可以丰富推荐结果，但不应改变前端依赖的核心字段，例如菜名、推荐理由、图片、步骤、餐厅地址和评分等。这样可以保证系统在不断增加能力的同时，仍然保持较低的联调成本。

另一个扩展方向是把当前后端一次性返回结果，升级为卡片级流式返回。系统可以先返回 3 个菜名和推荐理由，再逐个补充步骤、图片和来源。这样用户不会在空白页面等待全部任务结束，也更符合 Agent 多工具并发执行的真实过程。

如果进一步面向真实用户，还可以加入用户画像和营养分析。系统可以记录用户常见忌口、偏好菜系、预算范围和历史选择，并结合热量、蛋白质、油盐水平等信息给出更细的解释，使推荐从“好吃和方便”逐步扩展到“长期适合”。

6.2 下一阶段计划

- 第一阶段：完善日志和 Trace，把每个工具调用的输入、输出、耗时、失败原因记录下来，形成可回放的 Agent 调试链路，方便定位是模型、工具还是数据字段导致问题。
- 第二阶段：引入任务队列和缓存，对 Tavily 图片、小红书搜索、高德 POI 等结果做短期缓存，减少重复请求和等待时间，并为高并发场景预留扩展空间。
- 第三阶段：建设小型 Benchmark 数据集，覆盖辣、清淡、减脂、忌口、多人聚餐、附近餐厅等典型需求，用固定样例持续评估模型和工具链变化。
- 第四阶段：把用户确认、收藏、删除和修改结果写入长期记忆，使 Milvus 记忆不只是相似去重，还能真正影响下一次推荐，并逐步形成个性化饮食画像。

6.3 AI 能力演进路径

AI 能力演进不应只理解为更换更强模型。对这个项目来说，能力演进包括多模型路由、工具使用策略、上下文压缩、结构化校验和自动评测。简单问题可以交给快模型，复杂问题再调用深度推理模型；图片和来源由检索工具负责，步骤由小红书或 LLM 兜底；所有结果通过 schema 校验后再进入前端。

未来系统还可以根据任务复杂度自动选择不同策略。例如普通家常菜推荐使用低成本模型快速完成，涉及健康限制、多人偏好冲突或外出就餐比较时再启用更强推理模型和更多工具。这样既能控制成本，也能让复杂问题获得更充分的处理。

随着 AI 逐渐成为开发主力，系统也应从“人写代码、AI 辅助补全”演进为“人定义目标和约束、AI 生成方案、工程师验证和收敛”。这要求项目本身具备更好的可观测性和可测试性，否则 AI 生成的改动很难判断是否真正改善了系统。

因此，本项目后续升级的重点不只是增加功能数量，还包括让每一次推荐都更容易解释、复现和评估。只有当日志、测试样例、结构化字段和降级路径都比较清晰时，AI 生成或修改的代码才更容易被工程化吸收。

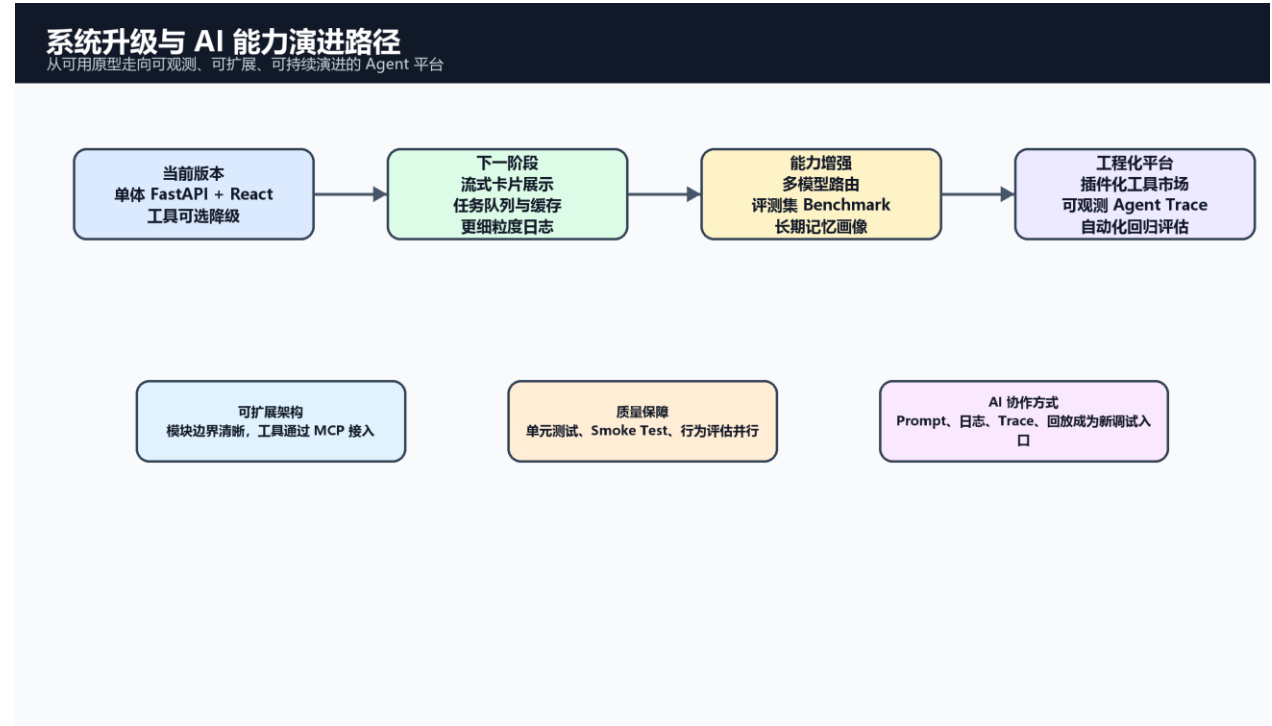


图 11 系统升级与 AI 能力演进路径

七、课程总结

7.1 个人收获

通过本项目，我对 Agentic AI 原生开发的理解从“会调用模型”转向“会组织一个由模型、工具、数据和界面共同构成的软件系统”。模型能力很重要，但工程边界更重要。一个可交付系统必须处理异常、延迟、字段兼容、用户体验、测试评估和文档沉淀。

另一个收获是认识到规格文档并不是开发前的形式任务，而是后续重构和调试的依据。当我们发现一次性生成完整菜谱容易失败时，能够快速回到 Product Spec 和 Architecture Spec，重新定义第一轮只生成候选菜名，第二轮并发补详情。这种调整不是简单改 Prompt，而是对系统数据流的重新设计。

7.2 工程思维转变

AI 逐渐变成开发主力后，工程师的工作重点会发生变化。过去主要关注代码是否能编译、接口是否能跑通；现在还要关注 Prompt 是否稳定、工具调用是否可观测、模型输出是否可验证、失败样例是否能复现。调试方式也从单纯断点调试，扩展到日志追踪、上下文回放、模型对照实验、结构化输出校验和 Benchmark 回归。

这种变化要求工程师具备更强的系统判断力。AI 可以快速生成代码和方案，但它并不天然知道项目的真实约束，也不会自动承担质量责任。工程师需要定义清晰目标、拆分可验证任务、判断方案是否符合架构边界，并在测试和日志中确认结果。换句话说，AI 提升了开发速度，但也放大了工程判断的重要性。

7.3 对课程的建议

课程可以进一步增加 Agent 调试、评测和工程化部署的内容。很多同学已经能够完成模型调用和页面展示，但真正困难的是如何发现模型失败、如何设计降级、如何建立评测样例、如何让 AI 参与代码开发但不破坏系统质量。如果课程中能加入更多关于 Agent Trace、工具调用日志、结构化输出评估和多模型对比的实践，会更贴近未来的软件开发方式。

总体来看，本项目完成了从需求分析、规格设计、架构实现、工具集成、测试评估到报告交付的完整闭环。它不是一个只展示模型能力的 Demo，而是一个尝试把 AI 能力纳入企业级软件工程流程的实践项目。